

Client-server, HTTP/HTTPS, DNS, DevTools

Головна ідея: коли ти вводиш адресу сайту в браузер, відбувається ланцюжок дій: браузер (клієнт) з'ясовує через DNS, де знаходиться потрібний сервер, встановлює з'єднання за протоколом HTTP/HTTPS, надсилає запит і отримує відповідь. Усі ці кроки можна побачити наочно у вкладці Network в DevTools браузера.

1. Client-server model (клієнт-серверна модель)

Це базова архітектура, на якій побудований весь веб: два типи учасників постійно обмінюються даними.

Роль	Що це	Приклад
Клієнт (client)	Програма, яка надсилає запит і відображає результат користувачу	Браузер (Chrome, Firefox), мобільний застосунок
Сервер (server)	Комп'ютер/програма, що постійно "слухає" запити, обробляє їх і надсилає відповідь	Веб-сервер, що віддає HTML-сторінку або дані з бази

Загальний цикл взаємодії

- Клієнт формує запит (наприклад, "дай мені сторінку /home")
- Запит через інтернет доходить до сервера
- Сервер, за потреби, звертається до бази даних, щоб зібрати потрібні дані
- Сервер формує відповідь і надсилає її назад клієнту
- Клієнт (браузер) отримує відповідь і відображає її як вебсторінку

Аналогія: це схоже на відправлення листа -- ти пишеш листа (запит), пошта (інтернет) доставляє його адресату (серверу), той пише відповідь і надсилає назад, а ти її читаєш (браузер відображає сторінку).

2. DNS (Domain Name System)

Комп'ютери в інтернеті звертаються один до одного за **IP-адресами** (наприклад, `142.250.186.78`) -- це як номер будинку. Але людям зручніше запам'ятовувати назви (`google.com`), а не набори цифр. DNS -- це система, що перекладає доменні імена в IP-адреси.

Як працює DNS-запит (спрощено)

- Ти вводиш у браузер `example.com`
- Браузер перевіряє свій кеш -- можливо, IP-адреса вже відома з попереднього разу
- Якщо ні -- запит іде до DNS-резолвера (зазвичай від провайдера інтернету)
- Резолвер звертається по ланцюжку DNS-серверів (root -> .com -> example.com), поки не знайде потрібну IP-адресу
- IP-адреса повертається браузеру, і той нарешті може встановити з'єднання із сервером за цією адресою

Термін	Значення
Домен (domain)	зрозуміле людині ім'я сайту, наприклад <code>github.com</code>
IP-адреса	числова адреса сервера в мережі, наприклад <code>140.82.121.3</code>
DNS-резолвер	сервер, що виконує "переклад" домену в IP-адресу
DNS-кеш	тимчасове збереження вже знайдених відповідностей домен -> IP, щоб не робити запит щоразу

Аналогія: DNS -- це як телефонна книга або контакти в телефоні: ти набираєш ім'я людини ("Мама"), а телефон сам підставляє потрібний номер, який ти навряд чи пам'ятаєш напам'ять.

3. HTTP / HTTPS

HTTP (HyperText Transfer Protocol) -- протокол (набір правил), за яким клієнт і сервер "спілкуються" між собою в вебі: як саме формулювати запит і як відповідати на нього.

Структура HTTP-запиту

Елемент	Приклад	Пояснення
Метод (method)	<code>GET</code> , <code>POST</code> , <code>PUT</code> , <code>DELETE</code>	яку дію хоче виконати клієнт (отримати дані / надіслати дані / оновити / видалити)
URL	<code>/api/users/5</code>	адреса ресурсу на сервері, до якого звертаються
Заголовки (headers)	<code>Content-Type: application/json</code>	додаткова інформація про запит (тип даних, авторизація тощо)
Тіло (body)	<code>{"name": "Ivan"}</code>	дані, які клієнт надсилає серверу (не завжди присутнє, наприклад у GET його зазвичай немає)

Структура HTTP-відповіді

Елемент	Приклад	Пояснення
Статус-код (status code)	<code>200</code> , <code>404</code> , <code>500</code>	число, що показує результат обробки запиту
Заголовки відповіді	<code>Content-Type: text/html</code>	метадані про те, що саме повертається
Тіло відповіді	HTML-код сторінки / JSON з даними	власне вміст, який відображає браузер

Найпоширеніші статус-коди

Код	Група	Значення
200 OK	2xx -- успіх	запит успішно оброблено
301 / 302	3xx -- редирект	ресурс переміщено, потрібно перейти за іншою адресою
400 Bad Request	4xx -- помилка клієнта	запит некоректний (наприклад, неправильні дані)
401 / 403	4xx -- помилка клієнта	немає авторизації / доступу заборонено
404 Not Found	4xx -- помилка клієнта	ресурс за такою адресою не знайдено
500 Internal Server Error	5xx -- помилка сервера	щось пішло не так на стороні сервера

HTTP vs HTTPS

HTTPS -- це той самий HTTP, але з додатковим шаром шифрування (**TLS/SSL**). Дані між клієнтом і сервером шифруються, тому їх не можна прочитати чи підмінити по дорозі.

	HTTP	HTTPS
Шифрування	немає -- дані передаються відкритим текстом	є -- дані зашифровані (TLS/SSL)
Порт за замовчуванням	80	443
Індикатор у браузері	попередження "Not Secure"	іконка замка  поруч з адресою
Коли використовувати	практично ніколи для реальних сайтів	стандарт для будь-якого сучасного сайту (особливо з формами, логінами, оплатою)

4. Повний життєвий цикл запиту (client-server + DNS + HTTP)

- 1

Користувач вводить **https://example.com** в адресний рядок браузера
- 2

Браузер перевіряє DNS-кеш, або звертається до DNS-резолвера, щоб отримати IP-адресу сервера
- 3

Браузер встановлює TCP-з'єднання із сервером за отриманою IP-адресою (порт 443 для HTTPS)
- 4

Відбувається TLS-"рукошлякування" (handshake) -- узгодження шифрування для HTTPS
- 5

Браузер надсилає HTTP-запит (метод, URL, заголовки)
- 6

Сервер обробляє запит (за потреби звертається до бази даних)
- 7

Сервер надсилає HTTP-відповідь (статус-код, заголовки, тіло -- HTML/CSS/JS/JSON)
- 8

Браузер отримує HTML і починає його рендерити (побудова DOM)
- 9

Під час рендерингу браузер знаходить посилання на CSS, JS, зображення -> надсилає додаткові запити на кожен ресурс (кроки 5--7 повторюються)
- 10

Сторінка повністю завантажена і відображена користувачу

Саме цей ланцюжок варто зобразити як діаграму для практичного завдання -- клієнт, DNS-резолвер і сервер як окремі "вузли", а стрілки між ними підписані відповідними кроками (DNS-запит, TCP/TLS, HTTP-запит, HTTP-відповідь).

5. Browser DevTools

DevTools (інструменти розробника) -- вбудована в браузер панель для перегляду й діагностики того, що відбувається "під капотом" сторінки. Відкривається клавішею **F12** або **Ctrl+Shift+I** (Windows/Linux), **Cmd+Option+I** (macOS).

Основні вкладки DevTools

Вкладка	Що показує / для чого потрібна
Elements	структура HTML (DOM) і CSS-стилі елементів сторінки в реальному часі -- можна редагувати "на льоту" для перевірки ідей
Console	вивід JavaScript-помилкок, попереджень, і місце, куди можна вручну вводити JS-код для тестування
Network	список усіх HTTP-запитів сторінки: метод, URL, статус-код, час завантаження, розмір, заголовки й тіло кожного запиту/відповіді
Application	перегляд сховищ браузера: cookies, localStorage, sessionStorage, кеш
Sources	перегляд і дебаг JS-файлів сторінки, встановлення breakpoints

Вкладка Network -- як побачити цикл запиту наживо

1. Відкрити DevTools -> Network -> перезавантажити сторінку (**Ctrl+R** / **Cmd+R**)
2. У списку з'являться усі запити -- HTML-документ, CSS, JS, зображення, API-запити тощо
3. Клік на будь-який запит показує: **Headers** (заголовки запиту й відповіді, статус-код, метод), **Preview/Response** (тіло відповіді), **Timing** (скільки часу зайняв кожен етап: DNS lookup, TCP, очікування відповіді сервера тощо)

Практична порада: у вкладці Network можна побачити реальний час, витрачений саме на DNS lookup і на очікування відповіді сервера (waiting/TTFB) -- це напряму ілюструє кроки життєвого циклу запиту з розділу 4.

Найкорисніші комбінації клавіш DevTools

Дія	Windows/Linux	macOS
Відкрити DevTools	F12 або Ctrl+Shift+I	Cmd+Option+I
Відкрити одразу Console	Ctrl+Shift+J	Cmd+Option+J
Інспектор елементів (обрати елемент на сторінці)	Ctrl+Shift+C	Cmd+Shift+C

6. Список ключових понять теми

Термін	Коротко
Client (клієнт)	програма, що надсилає запит (браузер, застосунок)
Server (сервер)	програма/машина, що обробляє запит і повертає відповідь
DNS	система, що перетворює домен (назву) на IP-адресу сервера
IP-адреса	числова адреса пристрою в мережі
HTTP	протокол обміну даними між клієнтом і сервером (незашифрований)
HTTPS	HTTP + шифрування (TLS/SSL)
Метод запиту	GET / POST / PUT / DELETE та ін. -- яку дію виконує запит
Статус-код	число у відповіді сервера, що показує результат (200, 404, 500 тощо)
DevTools	вбудовані в браузер інструменти для перегляду HTML/CSS/JS/мережі сторінки
DOM	структуроване представлення HTML-сторінки, яке будує браузер і з яким можна взаємодіяти через JS